AIC 2026 · 算法应用赛 · 视觉逻辑算法应用赛

# 圆形铁片四缺陷识别与分拣

## 基于自建 Gazebo 数字孪生的视觉 — 决策 — 控制闭环

**GXU RobotZ · 广西大学**

技术文档 v1.0 · 2026 年 11 月

ROS 2 Humble · Ignition Fortress · BehaviorTree.CPP 4.10 · 传统几何视觉

02

# 目录

五章：从平台对标到实测证据


01 **方案总览** 四层架构、话题契约与三个核心指标

02 **平台对标说明** 逐条对应官方规则，说明自建平台的等价性

03 **关键算法** 铁片定位、四类缺陷判据、图像集

04 **系统实现** 行为树决策、三轴控制、全过程可视化与实时判分

05 **验证与展望** 主结果、泛化回归、负向注入与后续计划

03

# 方案总览

四层解耦：仿真提供真值，视觉判型，决策调度，控制执行

### 仿真 Gazebo

Ignition Fortress 数字孪生： 12 块物料、 4 列 × 3 层放置货架、三轴龙门 + 吸盘、固定相机；物理与渲染均为真实计算

### 视觉 aic_vision_trad

OpenCV 传统几何：整图 Hough + 内亮外暗环对比 + Kasa 圆精修 + 四类缺陷判据（无训练数据依赖）

### 决策 aic_behavior_tree

BehaviorTree.CPP 4.10： 12 轮巡视、 GOTO/HANDLE 调度、视觉超时兜底与迟到结果防护

### 控制 gantry_controller

FIFO 队列 + 梯形速度剖面 + 关节闭环到位；上料走 set_pose 、取放走 DetachableJoint

**契约不变** /control/material_task · /vision/material_type · /sim/gantry_arrived · /sim/handle_done

**12 / 12**

识别与放置全对

真视觉、 0 超时

**60 位点**

随机几何 0 误判

5 个种子 × 12 块

**0.0 mm**

放置落位误差

单轮约 145 s

04

# 平台对标说明

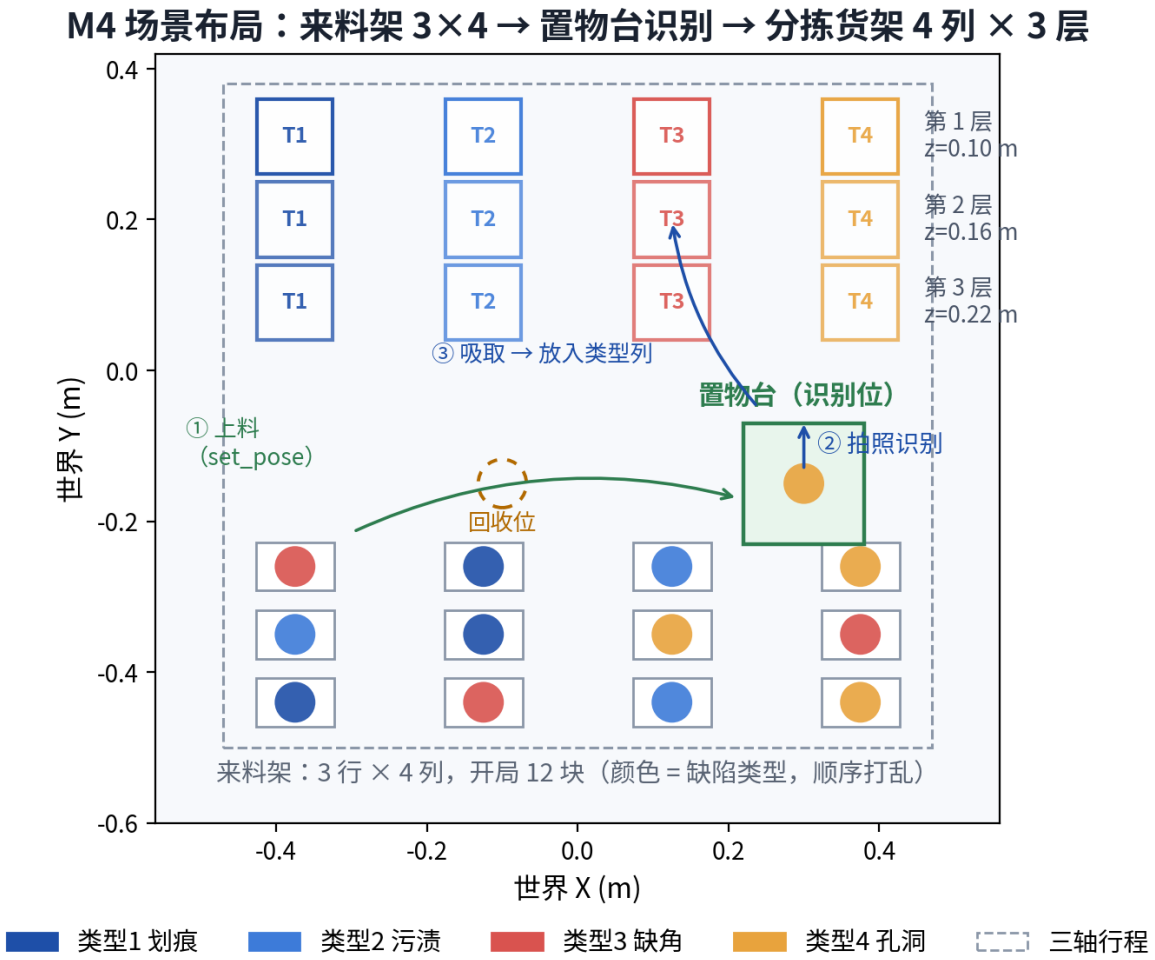
自建平台不是「替代品」：逐条实现官方规则，并用同一套指标验收

| 官方规则 / 能力 | 本方案实现 | 验收证据 |
| --- | --- | --- |
| 置物台生成物料 → 拍照 → 抓取 → 放置 | 每轮 set_pose 把 material_NN 传送到置物台，固定相机拍照后下探吸取 | 上料 12 次 OK；吸附 / 释放各 12 次 |
| 放置位四列对应四类缺陷，每列三个位置 | 货架 4 列 × 3 层，按类型列的最低空层放入，每格一块 | 12 块全部落对应列， L1→L3 逐层填满 |
| 每个库位有且只有一个物料（不堆叠） | 控制器维护每列已用层数，放满即拒绝并报错 | 落位断言：每格唯一， z 与层高一致 |
| 三轴行程 X/Y/Z = 300/300/200 mm | x ±0.45 m 、 y −0.24…+0.32 m 、 z −0.20…0 m | 行程覆盖来料架、置物台、最远层 |
| 吸盘吸取（ SuctionEnable ） | 吸盘几何 + DetachableJoint 吸附 / 断气 | 吸附与释放成对，物料随龙门移动 |
| 自动运行、正确变绿 / 错误变红、自动计分 | 行为树自动 12 轮；看板按类型列判分，绿 / 红 / 琥珀实时显示 | 单轮约 145 s 无人干预；看板 12 绿 |

结论：官方平台的四项硬指标（上料节拍、四列三位置、吸盘取放、自动计分）在自建平台上均可复现，且每一步都有可回放的日志与位姿证据。

# 05 任务与场景

布局由单一 YAML 驱动：改一处，世界、控制、判分同时生效




- **上料位**

  来料架 3×4 开局铺满 12 块；每轮 set_pose 传送到置物台 (0.30, −0.15) 识别

- **放置货架**

  4 列 = 四类缺陷；每列 3 层，层位沿 +Y 阶梯错开 0.11 m，避免吸盘升降撞上层托盘

- **识别相机**

  固定在置物台正上方，先拍照后下探，吸盘不遮挡光路

- **坐标口径**

  +X 向右（列 1→4），+Y 向远，+Z 向上；与 MaterialTask.frame_id 一致

06

# 关键算法 · 铁片定位

先找「盘在哪」，再判「盘上是什么」——定位精度决定所有归一化判据

1

### 整图 Hough 圆检测

不要求盘居中；对浅色台面 / 深色地贴都成立

2

### 内亮外暗环对比选优

用盘内外亮度差挑出真正的盘，排除孔洞与阴影误检

3

### 径向扫描 + Kasa 拟合

180 条射线 × 40 个半径，取边缘内点做最小二乘圆拟合

4

### 失败兜底

整图失败时退回 ROI 拟合圆；两条路径都过同一套精修

定位失败时「不发消息」而非猜一个类型——决策侧记 TIMEOUT

**0.1–0.5 px**

精修后半径抖动

Hough 原始 ±9%

**85 px**

盘半径（640×480）

距盘面约 0.26 m

### 为什么必须精修

孔洞半径、划痕长度都按「盘半径」归一化。 Hough 的 dp=1.2 让半径落在 1.2 px 网格上，逐帧抖动 80.3–87.4 px；同一块料的 frac 会从 0.030 漂到 0.072，阈值无从下手。

精修后组内标准差 0.10–0.49 px。破损不依赖半径：改用「填充率」 fill （0.26 vs 阈值 0.62），随机化后仍稳。

07

# 关键算法 · 四类缺陷判据

几何量优先，几何失效时换物理量——判据链按「误判代价」排序

| 类型 | 主判据 | 阈值 | 实测余量 |
| --- | --- | --- | --- |
| 4 孔洞 | 圆度 + 面积 + 亮度差；贴边时用「近黑核心」兜底 | circ ⩾ 0.70, 0.008 ⩽ frac ⩽ 0.045, drop ⩾ 100 | 孔 core 0.23–0.75，其余 0.000 |
| 1 划痕 | 细长比 + 圆度 | elong ⩾ 12, circ ⩽ 0.25 | 实测 elong 20.6 （划痕） |
| 3 破损 | 填充率（折线填不满外接框） + 长宽比 | fill ⩽ 0.62, elong ⩽ 6, 0.004 ⩽ frac ⩽ 0.060 | fill 0.26–0.27， 2.3 倍 |
| 2 污渍 | 以上都不满足的暗斑（兜底类） | — | 与破损以 fill 0.62 分界 |

**两个设计取舍**

① 阈值不贴实测值：破损 fill 实测 0.26–0.27，阈值取 0.62 （2.3 倍）；面积比再留 2.7 倍余量 —— 为「精修被拒、退回整像素圆」的最坏情况留口子。

② 贴边孔洞会伪装成其它类：连通域被侧壁阴影粘住后 circ 0.98→0.34 、 rad 0.78，几何量整体失效；改用「斑内近黑像素占比」这一物理量，放在破损判据之前。

08

# 图像集展示

4 类 × 3 例：每格标注判据复算值，可直接核对阈值余量




- **采集口径**

  全部来自 Gazebo 相机实时渲染，非预置图片；每帧 640×480 ，盘半径约 85 px

- **标注内容**

  类型、判据名与关键量（ fill / elong / circ / frac / drop / core ），由脚本复算而非手写

- **覆盖范围**

  四类各 3 例，含贴边孔洞、细长划痕等边界样本

- **用途**

  作为阈值回归基线；几何随机化后同口径可再生成图版对比

图版由 scripts/make_atlas.py 从调试图重排生成：脚本会重新调用分类器，因此图上的判据值与当时的判定结果必然一致，不存在「图注与代码不同步」。

09

# 系统实现 · 行为树决策

12 轮巡视是固定的，异常才是变量——兜底逻辑比主流程更重要




### 视觉超时兜底

决策对 GOTO/HANDLE 都递增 visit_index，不能用编号推相邻位点。改为语义推断：新 GOTO 到达时，仍停在「识别中」的位点记 TIMEOUT。

### 迟到结果防护

识别结果晚到会被下一位点误收（类型错位但结果表仍显示 OK）。窗口按最坏队列时延设定： M4 实测 GOTO→ 结果 15.5 s，现取 30 s。

### 为什么用行为树

巡视、识别、取放三类动作的失败语义完全不同：识别失败要跳过、取放失败要记录、通信失败要重试。用树把「正常序列」与「异常分支」分开，决策层不含任何运动学细节，控制层可整体替换为真机 PLCnext 而不改一个节点。

# 10 系统实现 • 三轴控制

指令是 fire-and-forget，执行必须自己保证「真的到位」

1. 上料： set_pose 传送到置物台
2. 走到停靠偏移位（吸盘让出光路）
3. 下探 → 吸盘吸附（ DetachableJoint ）
4. 抬升 → 平移到类型列最低空层
5. 触底放置 → 断气 → 抬升 → 完成

单次 HANDLE 实测 12.8 s： 取料 + 搬运 + 放入三个动作

**0–1.2 mm**

稳态到位误差

闭环判据

**3 / 10 mm/s**

位置 / 速度阈值

连续 3 拍确认

### 三个必须写进代码的细节

① 到位 = 反馈收敛，不是剖面走完。早期按「剖面时间 + 0.15 s 」报到位，实际还在追、过冲 17.7 mm ，相机偏一整格。

② 每拍都下发目标。竖直轴是速度伺服，停发后重力把它拉到底（实测掉 420 mm ）。

③ 放置用「触底」判据：托盘接触后控制器仍在下压，按精确到位判会白等到超时，并连锁挤爆下一个位点的视觉窗口。

# 11 全过程可视化与实时判分

一页看全：抓取过程、识别结果、得分——单窗口即可录制



固定相机实拍：置物台上的铁片与划痕（640×480）

- **全景 · 自由摄像机**

  世界内独立相机，看板可切 4 个预设机位并六向平移，抓取全程可见

- **末端相机画中画**

  识别原图 + 当前位点 / 判型 / 实时比分

- **俯视态势图**

  物料从来料架 → 置物台 → 吸盘（跟随龙门）→ 货架层位，全程位置可见

**为什么判分必须独立成节点**

判分只订阅、不发布：它挂了链路照常跑，它算错也不会反向影响决策。真值只取 sim_layout.yaml 的 materials.types 一份事实，避免「视觉真值」与「判分真值」漂移。

浏览器打开 127.0.0.1:8080 ， OBS 录这一个窗口即满足「单窗口合成录制」。

# 12 实验与验证

所有结论都可在本机复现：一条命令跑完并自动断言

**12 / 12**
主结果全对
真视觉、 0 超时

**0 误判**
60 位点泛化
5 种子 × 12 块

**0.0 mm**
落位误差
x / y 方向

**12 + 12**
吸附 / 释放
无卡料、无漏放

| 验证项 | 方法 | 结果 |
| --- | --- | --- |
| 识别正确性 | 真视觉跑完整一轮，逐位点比对 materials.types | 12/12 ， 0 超时 |
| 阈值泛化 | jitter_seed 1–5 随机化缺陷几何，各跑 12 位点 | 60 位点 0 误判 |
| 物理落位 | pose/info 读 12 块物料世界坐标，对照类型列与层高 | 误差 0.0 mm ， L1–L3 逐层填满 |
| 鲁棒性 | 注入 60% 识别失败（ drop_rate=0.6 ） | 巡视不中断，超时被正确计数 |
| 关停正确性 | 整轮后检查吸附 / 释放条数 | 各 12 次，最后一块不卡在吸盘上 |

一键复现： ./scripts/verify_sim_m2.sh （跑完自动调 check_sim_m2.py 做三组断言，失败打印具体条目）

# 13 总结与展望

省赛要件已齐备；国赛把「能跑通」升级为「可迁移」

**合规** 官方四项硬指标逐条实现：上料节拍、四列三位置、吸盘取放、自动计分

**可靠** 12/12 、 60 位点 0 误判、 0.0 mm 落位；超时与迟到结果都有显式防护

**完整** 识别 → 决策 → 取放 → 判分全链路可看、可录、可复现，单条命令自验

## 下一步

- 复合缺陷多标签：同一块料同时有划痕 + 孔洞时的判据优先级
- 赛题映射表：把官方评分细则逐条映射到代码与日志字段
- 真机迁移：控制层替换为 PLCnext ，视觉与决策零改动
- 录制与提交：按官方要求完成一镜到底录像与命名

技术文档与仿真平台均在本仓库内，可按 README 一键构建复现。